Republic of the Philippines
Bicol University Polangui
Polangui Albay

LAB REPORT II

SUBMITTED BY:

GIAN CARLO TRILLES
BScPE-2A

SUBMIITED TO:

MELISSA JEANKIE RELLON
INSTRUCTOR

DATE:
OCTOBER 10,2024

MACHINE PROBLEM DESCRIPTION:

This Python program illustrates the nature of machine IDs with the help of five sorting algorithms: Bubble Sort, Selection Sort, Insertion Sort, Quick Sort, and Merge Sort.

BUBBLE SORT:

This is an algorithm that compares adjacent elements and if they are in incorrect order then exchanges them. It keeps repeating this way until the whole list becomes sorted; it's called "bubbling up" where all the largest elements end up after each pass in their correct positions.

SELECTION SORT:

This algorithm repeatedly chooses the smallest element from the unsorted portion of the list and during each pass swaps it with the current element. It works by placing each smallest element into its correct position along the way, starting at the beginning of the list.

INSERTION SORT:

This algorithm sorts the list constructing an ordered part from one element at a time. It repeatedly takes the next unsorted element and inserts it in its correct position in the ordered part.

QUICK SORT:

It employs a divide-and-conquer approach to sort the list. The algorithm selects a pivot element and divides the array around the pivot and recursively sorts sub-arrays on both sides of the pivot.

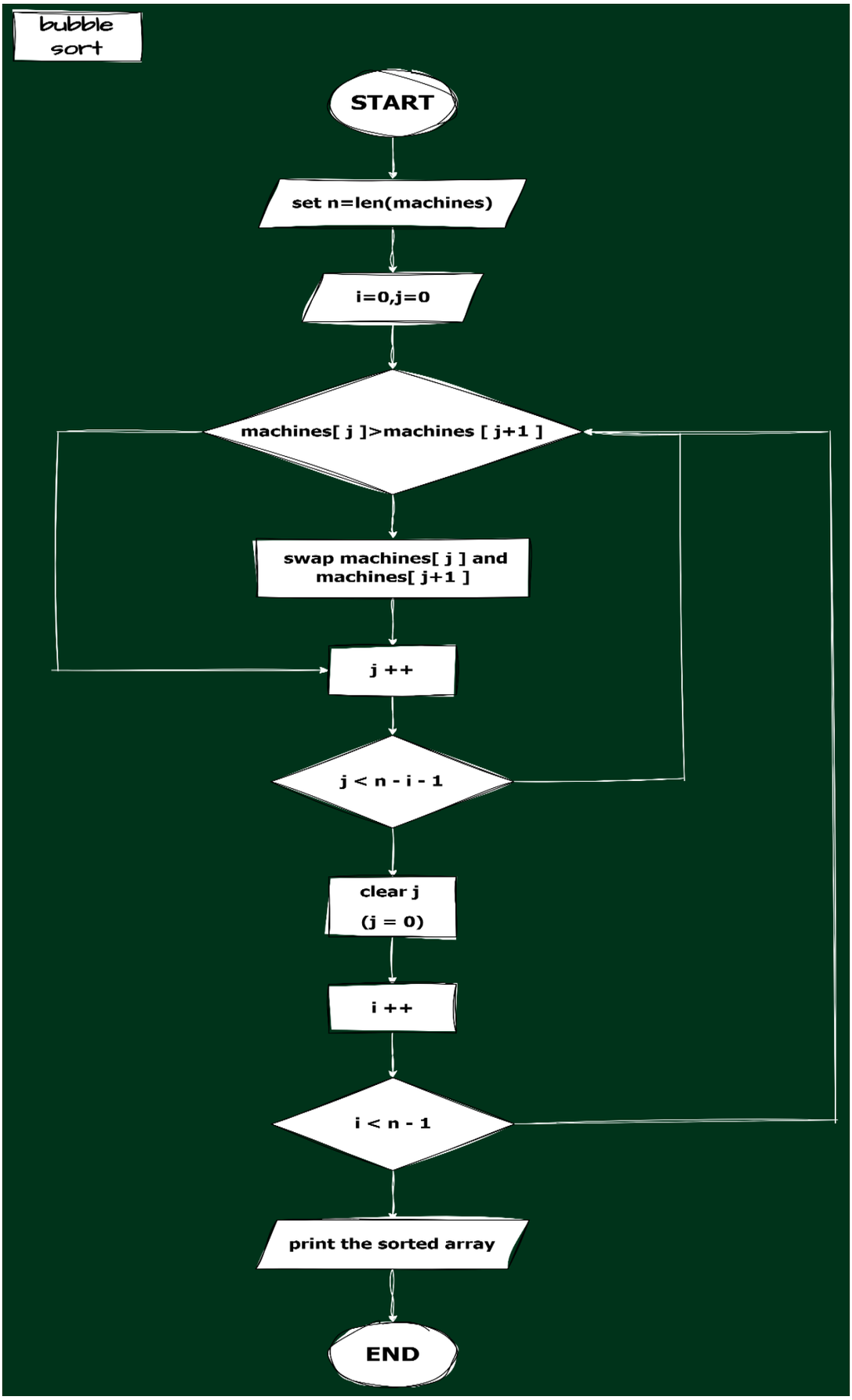
MERGE SORT:

This is a divide-and-conquer algorithm that splits the array into two halves and recursively sorts both halves. After this, merge the two sorted halves together.

It sorts the machine IDs, then applies the XOR operation to determine which specific machine ID occurs only once in the list while all other machine IDs occur twice.

I FLOWCHART (BUBBLE SORT):

In the following flowchart, the program uses Bubble Sort on a single sequence of machine IDs. Then it displays the steps for repeatedly comparing adjacent elements and swapping them if they do not have the ascending order until the end of the whole array.



SOURCE CODE:

The code below demonstrates the application of the Bubble Sort algorithms on an array of machine IDs. It loops over the array, sorting elements with one of the algorithms, and then printing out the final sorted array.

FaultyMachine_BubbleSorting.py

```python
#perform bubble sort in this function
def bubbleSort(machines):  1 usage
    n = len(machines)
    for i in range(n):
        for j in range(0, n-i-1):
            if machines[j] > machines[j+1]:
                machines[j], machines[j+1] = machines[j+1], machines[j]
    return machines

def findFaultyMachine(machines):  1 usage
    result = 0
    for machine in machines:
        result ^= machine
    return result

def main():  1 usage
    machines = [101, 202, 303, 404, 101, 303, 404]

    # use for sortinf the machine ID
    sorted_machines = bubbleSort(machines)
    print(f"Sorted machines using Bubble Sort: {sorted_machines}")


    faultyMachine = findFaultyMachine(sorted_machines)
    print(f"The faulty machine is: {faultyMachine}")

if __name__ == "__main__":
    main()
```

OUTPUT:

In this program, the machine IDs are sorted in ascending order, then printed as an array.

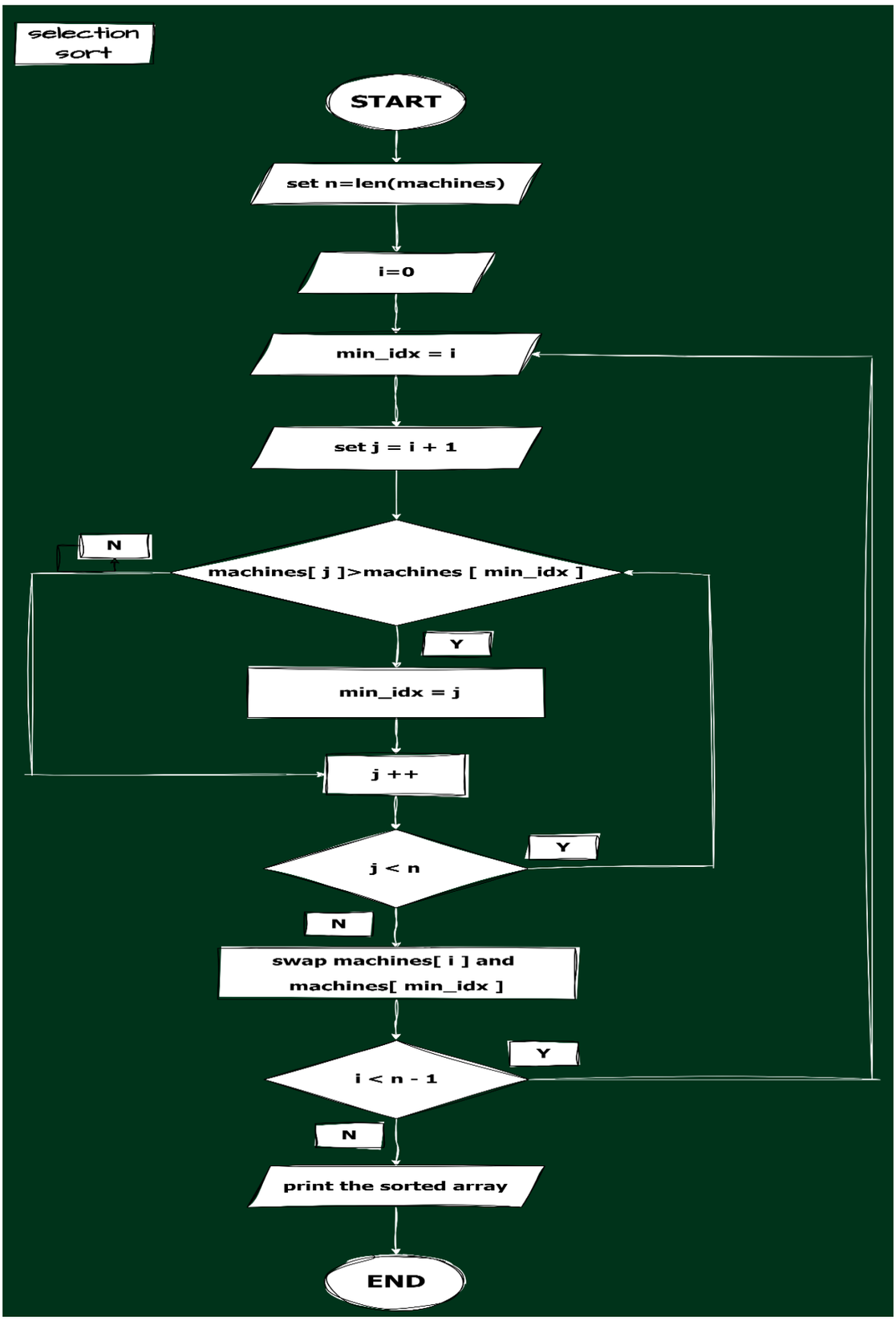
Run FaultyMachine_BubbleSorting

```
"C:\Users\Gian Carlo Trilles\PycharmProjects\pythonProject\.venv\Scripts\py
Sorted machines using Bubble Sort: [101, 101, 202, 303, 303, 404, 404]
The faulty machine is: 202

Process finished with exit code 0
```

II FLOWCHART (SELECTION SORT):

This flow chart demonstrates how the algorithm would carry out one pass over the machine IDs with the Selection Sort technique. It will cycle through the array and choose the smallest element remaining in the unsorted portion of the array then exchange it with the current element. This process is repeated to demonstrate how the selected element is then moved until all elements are sorted into ascending order as shown by the flow chart.



SOURCE CODE:

The code below demonstrates the application of the Selection Sort algorithms on an array of machine IDs. It loops over the array, sorting elements with one of the algorithms, and then printing out the final sorted array.

FaultyMachine_SelectionSorting.py

```python
#perform selection sort in this function
def selectionSort(machines):
    n = len(machines)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if machines[j] < machines[min_idx]:
                min_idx = j
        machines[i], machines[min_idx] = machines[min_idx], machines[i]
    return machines

def findFaultyMachine(machines):
    result = 0
    for machine in machines:
        result ^= machine
    return result
def main():
    machines = [101, 202, 303, 404, 101, 303, 404]

    # Sort the machine IDs using Selection Sort
    sorted_machines = selectionSort(machines)
    print(f"Sorted machines using Selection Sort: {sorted_machines}")

    faultyMachine = findFaultyMachine(sorted_machines)
    print(f"The faulty machine is: {faultyMachine}")

if __name__ == "__main__":
    main()
```

OUTPUT:

In this program, the machine IDs are sorted in ascending order, then printed as an array.

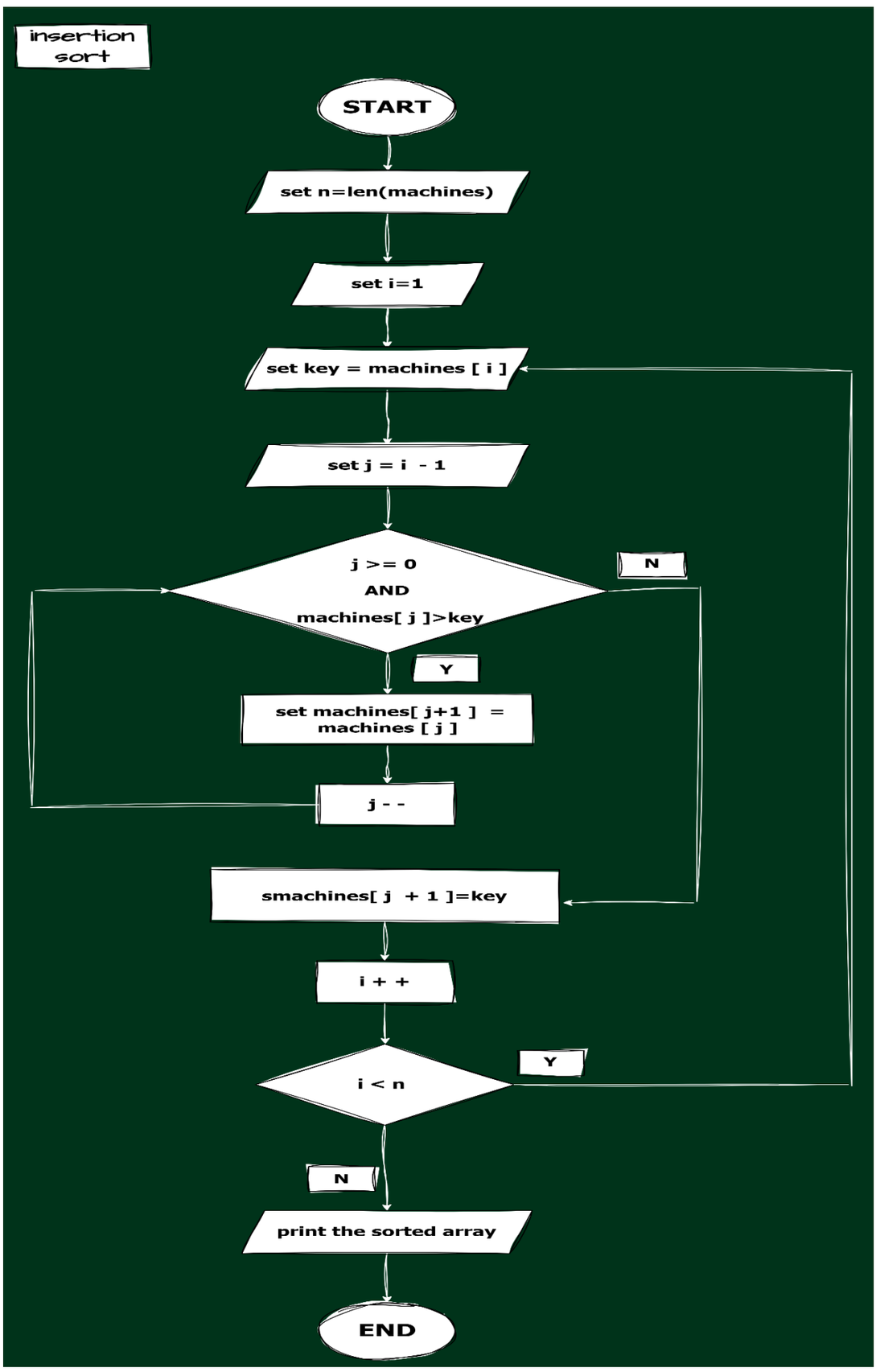
Run FaultyMachine_SelectionSorting

```
"C:\Users\Gian Carlo Trilles\PycharmProjects\pythonProject\.venv\Scripts\python.e
Sorted machines using Selection Sort: [101, 101, 202, 303, 303, 404, 404]
The faulty machine is: 202

Process finished with exit code 0
```

III FLOWCHART (INSERTION SORT):

This flowchart shows the scenario in which the algorithm is employing Insertion Sort to sort a single sequence of machine IDs. The algorithm successively constructs a sorted list by bringing in each of the unsorted elements in turn, and puts it in the right place in the sorted portion. In this flowchart, how the sorted part of the array increases as new elements successively come in in sorted order is illustrated.



SOURCE CODE:

Below is the application of the Insertion Sort algorithm on an array of machine IDs. The function insertionSort sorts the array by building a sorted portion one element at a time. It picks every element from the unsorted portion and places it at its proper location in the sorted part of the array. This process continues until all elements are sorted. After sorting, the code now goes to identify the malfunctioning machine by performing an XOR operation; because the XOR operation finds the only machine ID which occurs once.

FaultyMachine_InsertionSorting.py

```python
#perfor insertion sort in this function
def insertionSort(machines):  1 usage
    n = len(machines)
    for i in range(1, n):
        key = machines[i]
        j = i - 1
        while j >= 0 and machines[j] > key:
            machines[j + 1] = machines[j]
            j -= 1
        machines[j + 1] = key
    return machines
def findFaultyMachine(machines):  1 usage
    result = 0
    for machine in machines:
        result ^= machine
    return result
def main():  1 usage
    machines = [303, 101, 404, 202, 101, 303, 404]
    print(f"Original machines: {machines}")
    sorted_machines = insertionSort(machines)
    print(f"Sorted machines using Insertion Sort: {sorted_machines}")
    faultyMachine = findFaultyMachine(sorted_machines)
    print(f"The faulty machine is: {faultyMachine}")
if __name__ == "__main__":
    main()
```

OUTPUT:

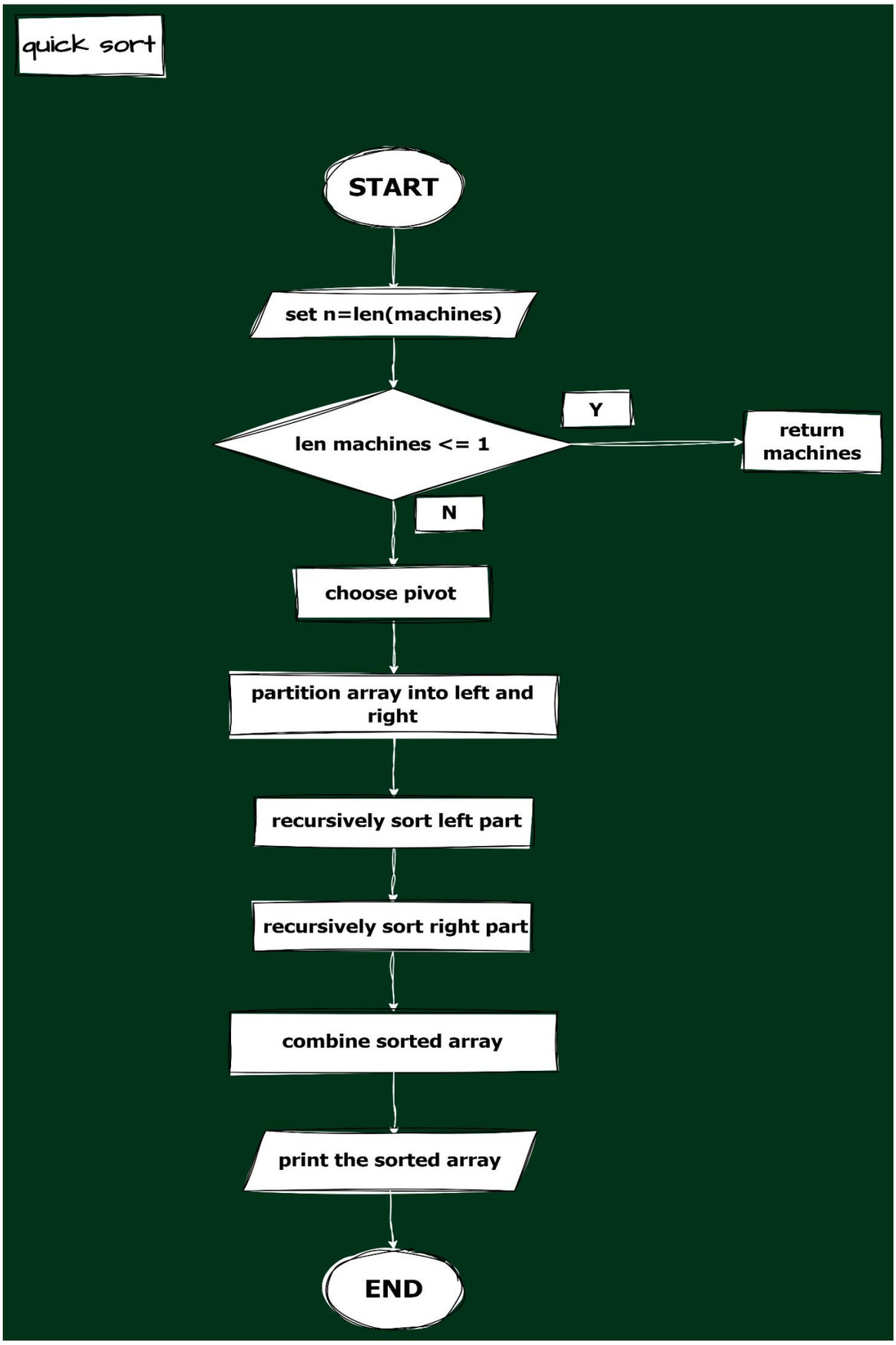
In this program, the machine IDs are sorted in ascending order, then printed as an array.

```
"C:\Users\Gian Carlo Trilles\PycharmProjects\pythonProject\.venv\Scripts\python.exe" "C:\
Original machines: [303, 101, 404, 202, 101, 303, 404]
Sorted machines using Insertion Sort: [101, 101, 202, 303, 303, 404, 404]
The faulty machine is: 202

Process finished with exit code 0
```

IV FLOWCHART (QUICK SORT):

This is just an illustration of sorting one list of machine IDs using Quick Sort. The fundamental idea behind the algorithm is the divide-and-conquer method. Choose a pivot and partition the data around the pivot. The flowchart shows how the list gets divided, breaking it down into two or more sublists, sorting each one separately, and then reassembling the list in the sorted order.



SOURCE CODE:

Below is the code that applies the Quick Sort algorithm on an array of machine IDs. The quickSort function applies a divide-and-conquer algorithm in which one element, the pivot, splits an array into two sub-arrays the left array with elements all less than the pivot and the right sub-array with elements larger than the pivot. The function then repeats itself recursively to the sub-arrays. After the array is sorted totally, then it uses the XOR operation by going through each machine ID and thereby selects the one ID which comes just once.

FaultyMachine_QuickSorting.py

```python
#perform quick sort in this function
def quickSort(machines):  3 usages
    if len(machines) <= 1:
        return machines
    else:
        pivot = machines[0]
        left = [x for x in machines[1:] if x <= pivot]
        right = [x for x in machines[1:] if x > pivot]
        return quickSort(left) + [pivot] + quickSort(right)

def findFaultyMachine(machines):  1 usage
    result = 0
    for machine in machines:
        result ^= machine
    return result

def main():  1 usage
    machines = [303, 101, 404, 202, 101, 303, 404]
    print(f"Original machines: {machines}")
    sorted_machines = quickSort(machines)
    print(f"Sorted machines using Quick Sort: {sorted_machines}")
    faultyMachine = findFaultyMachine(sorted_machines)
    print(f"The faulty machine is: {faultyMachine}")

if __name__ == "__main__":
    main()
```

OUTPUT:

In this program, the machine IDs are sorted in ascending order, then printed as an array.

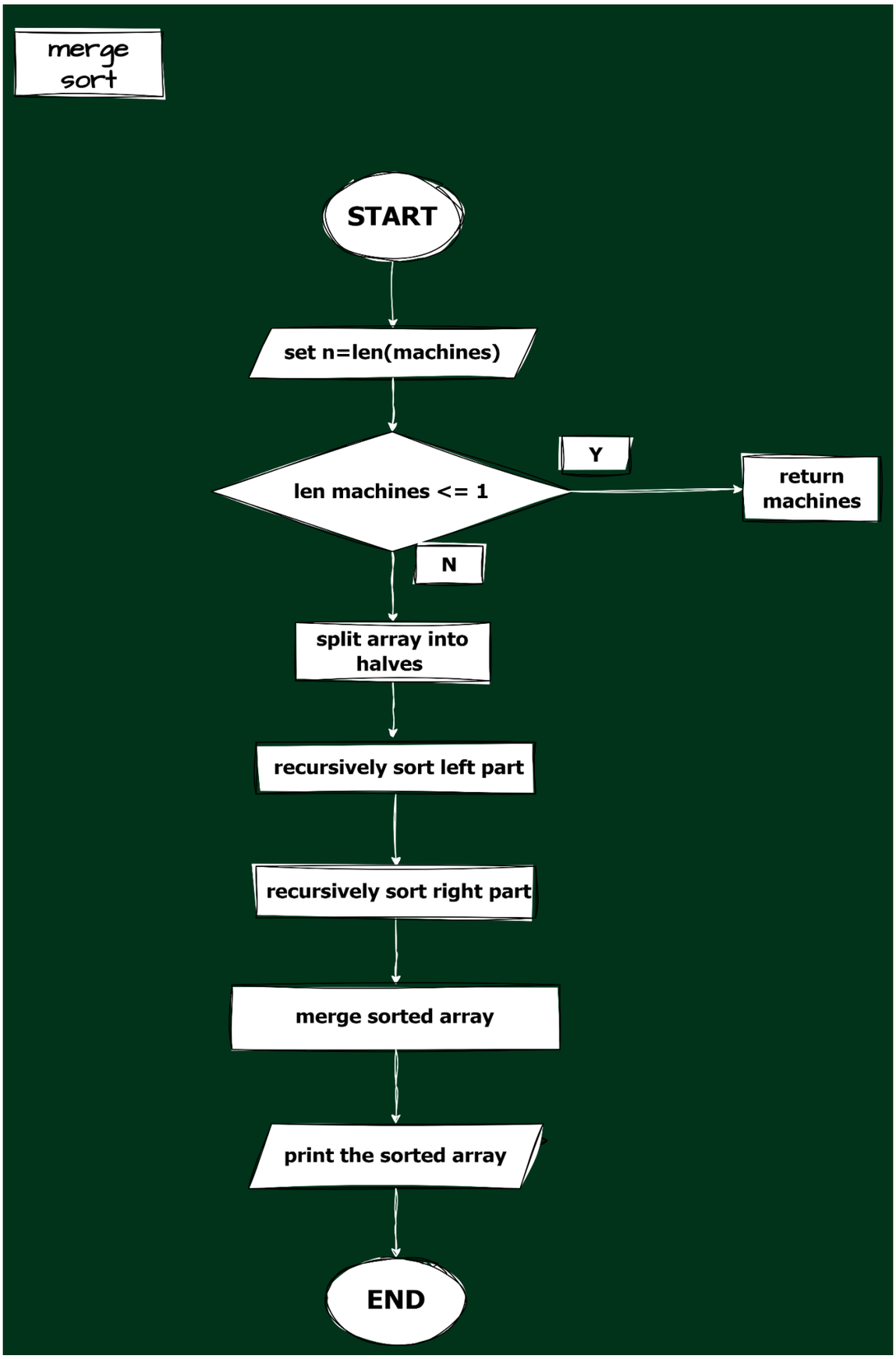
Run FaultyMachine_QuickSorting

```
"C:\Users\Gian Carlo Trilles\PycharmProjects\pythonProject\.venv\Scripts\python.exe" "C:
Original machines: [303, 101, 404, 202, 101, 303, 404]
Sorted machines using Quick Sort: [101, 101, 202, 303, 303, 404, 404]
The faulty machine is: 202

Process finished with exit code 0
```

V FLOWCHART (MERGE SORT):

This flowchart can be interpreted as processing a single set of machine IDs using the Merge Sort algorithm. The algorithm splits the array into two halves, recursively sorts each half, and then merges them together in sorted order. The flowchart illustrates how this process-the splitting and merging of an array step by step to produce the sorted array-can be done.



SOURCE CODE:

The following code implements the Merge Sort algorithm on a vector of machine IDs. The merge Sort method splits the vector into smaller vectors until each vector has only one element in it. Then the merge (left, right) method is called to combine the sorted vectors back into one single array in order. This process continues with recursion until all the elements of the array are sorted

FaultyMachine_MergeSorting.py

```python
#perform merge sort inthis function
def mergeSort(machines):  4 usages
    if len(machines) <= 1:
        return machines
    mid = len(machines) // 2
    left = mergeSort(machines[:mid])
    right = mergeSort(machines[mid:])
    return merge(left, right)
#combine two sorted array to single sorted array
def merge(left, right):  1 usage
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
def findFaultyMachine(machines):  1 usage
    result = 0
    for machine in machines:
        result ^= machine
    return result
def main():  1 usage
    machines = [303, 101, 404, 202, 101, 303, 404]
    print(f"Original machines: {machines}")
    sorted_machines = mergeSort(machines)
    # Sort the list of machines
    sorted_machines = mergeSort(machines)
    print(f"Sorted machines using merge sort: {sorted_machines}")
    faulty_machine = findFaultyMachine(machines)
    print(f"Faulty machine: {faulty_machine}")
if __name__ == "__main__":
    main()
```

OUTPUT:

In this program, the machine IDs are sorted in ascending order, then printed as an array.

Run FaultyMachine_MergeSorting

```
"C:\Users\Gian Carlo Trilles\PycharmProjects\pythonProject\.venv\Scripts\python.exe" "C:
Original machines: [303, 101, 404, 202, 101, 303, 404]
Sorted machines using merge sort: [101, 101, 202, 303, 303, 404, 404]
Faulty machine: 202

Process finished with exit code 0
```